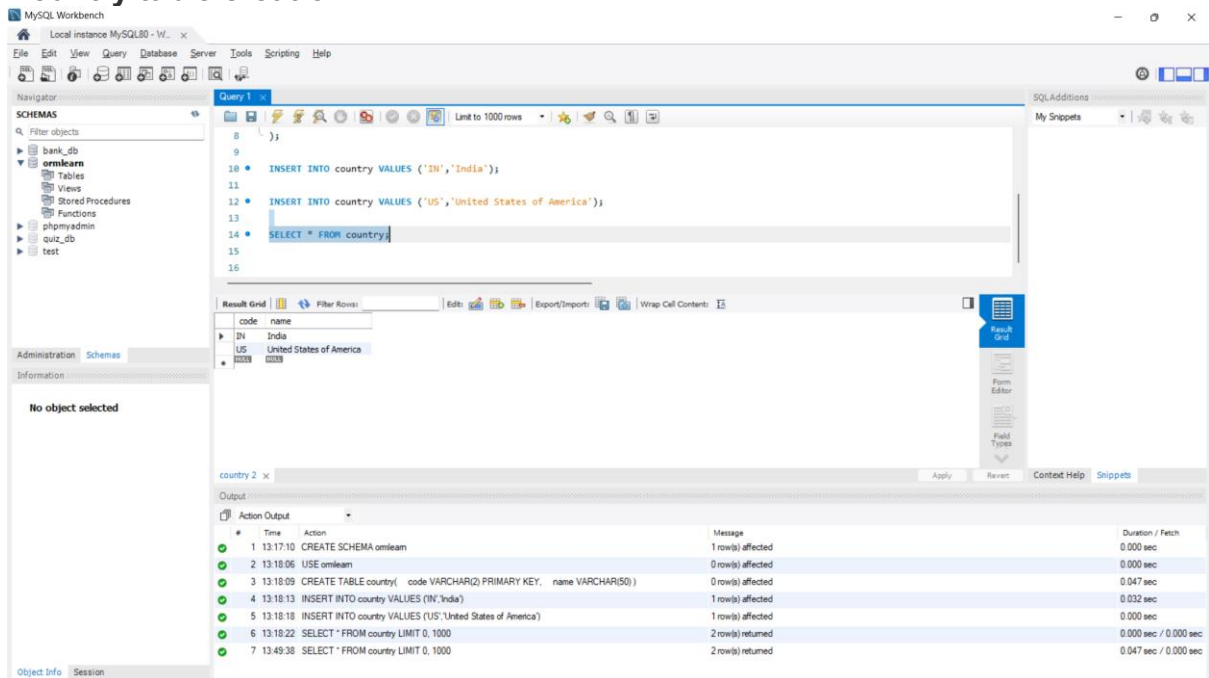


Spring Data JPA - Quick Example

Country table creation



Testing in OrmLearnApplication.java

```
package com.cognizant.ormlearn;
```

```
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
```

@SpringBootApplication

```
public class OrmLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;
    public static void main(String[] args) {
        ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);
        countryService =
            context.getBean(CountryService.class);
        LOGGER.info("Inside main");
        testGetAllCountries();
    }
}
```

```

    }

    private static void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> countries =
            countryService.getAllCountries();
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End");
    }
}

```

OUTPUT

The screenshot shows the Eclipse IDE with the following components:

- Package Explorer:** Shows the project structure with packages like `com.cognizant.ormlearn` and `com.cognizant.ormlearn.model`.
- Editor:** Displays the `CountryService.java` file with the following code:


```

28
29
30 countryService =
31     context.getBean(CountryService.class);
32
33 LOGGER.info("Inside main");
34
35 testGetAllCountries();
36
37 }
38
39
40 private static void testGetAllCountries() {
41
42     LOGGER.info("Start");
43
44     List<Country> countries =
45         countryService.getAllCountries();
46
47     LOGGER.debug("countries={}", countries);
48
49     LOGGER.info("End");
50
51 }
52

```
- Console:** Shows the application output, including database connection logs and the execution of the `testGetAllCountries` method. The output shows that the application successfully retrieved the country data (India) and logged it.


```

23-06-26 13:58:26.608 restartedMain INFO o.h.o.connections.pooling logConnectionInfo 163 HHH10001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 5.5.5
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
23-06-26 13:58:28.464 restartedMain INFO p.i.JtaPlatformInitiator initiateService 59 HHW000489: No JTA platform available (set
23-06-26 13:58:28.553 restartedMain INFO r.EntityManagerFactoryBean buildNativeEntityManagerFactory 447 Initialized JPA EntityManagerFactory
23-06-26 13:58:29.245 restartedMain INFO OptionalLiveReloadServer startServer 59 LiveReload server is running on port 35729
23-06-26 13:58:29.271 restartedMain INFO c.c.o.OrmLearnApplication logStarted 59 Started OrmLearnApplication in 10.477 seco
23-06-26 13:58:29.283 restartedMain INFO c.c.o.OrmLearnApplication main 32 Inside main
23-06-26 13:58:29.283 restartedMain INFO c.c.o.OrmLearnApplication testGetAllCountries 39 Start
23-06-26 13:58:29.492 restartedMain DEBUG org.hibernate.SQL logStatement 135 select c1.0.code,c1.0.name from country c1
23-06-26 13:58:29.804 restartedMain DEBUG c.c.o.OrmLearnApplication testGetAllCountries 44 countries=[Country [code=IN, name=India],
23-06-26 13:58:29.806 restartedMain INFO c.c.o.OrmLearnApplication testGetAllCountries 46 End
23-06-26 13:58:30.323 locationShutdownHook INFO r.EntityManagerFactoryBean destroy 460 Closing JPA EntityManagerFactory for persi
23-06-26 13:58:30.332 locationShutdownHook INFO c.z.h.HikariDataSource close 349 HikariPool-1 - Shutdown initiated...
23-06-26 13:58:30.349 locationShutdownHook INFO c.z.h.HikariDataSource close 351 HikariPool-1 - Shutdown completed.

```

Difference between JPA, Hibernate and Spring Data JPA

Java Persistence API (JPA)

JPA (Java Persistence API) is a specification that defines how Java objects are mapped to relational databases.

Features:

- Standard specification (JSR 338)
- Provides annotations such as `@Entity`, `@Table`, `@Id` and `@Column`
- Does not contain implementation code
- Requires an implementation provider

Examples of JPA Providers:

- Hibernate
- EclipseLink
- OpenJPA

Hibernate

Hibernate is an Object Relational Mapping (ORM) framework that implements the JPA specification.

Features:

- Provides actual implementation of JPA
- Converts Java objects into database records
- Handles CRUD operations
- Generates SQL queries automatically
- Supports caching and transaction management

Advantages:

- Reduces manual SQL coding
- Supports database portability
- Improves productivity

Spring Data JPA

Spring Data JPA is a framework built on top of JPA implementations such as Hibernate.

Features:

- Reduces boilerplate code
- Provides Repository interfaces
- Automatically generates CRUD methods
- Integrates with Spring Framework
- Simplifies transaction management

Advantages:

- Minimal code
- Faster development
- Better maintainability

Relationship Between Them

JPA → Specification

Hibernate → Implementation of JPA

Spring Data JPA → Abstraction layer over Hibernate/JPA

Application

↓

Spring Data JPA

↓

Hibernate

↓

JPA Specification

↓

Database

Conclusion

JPA is a specification that defines persistence standards. Hibernate is an implementation of JPA that performs ORM operations. Spring Data JPA is built on top of Hibernate and reduces boilerplate code by providing repository-based database access. Therefore, Spring Data JPA offers the simplest and most productive approach for database operations in Spring applications.